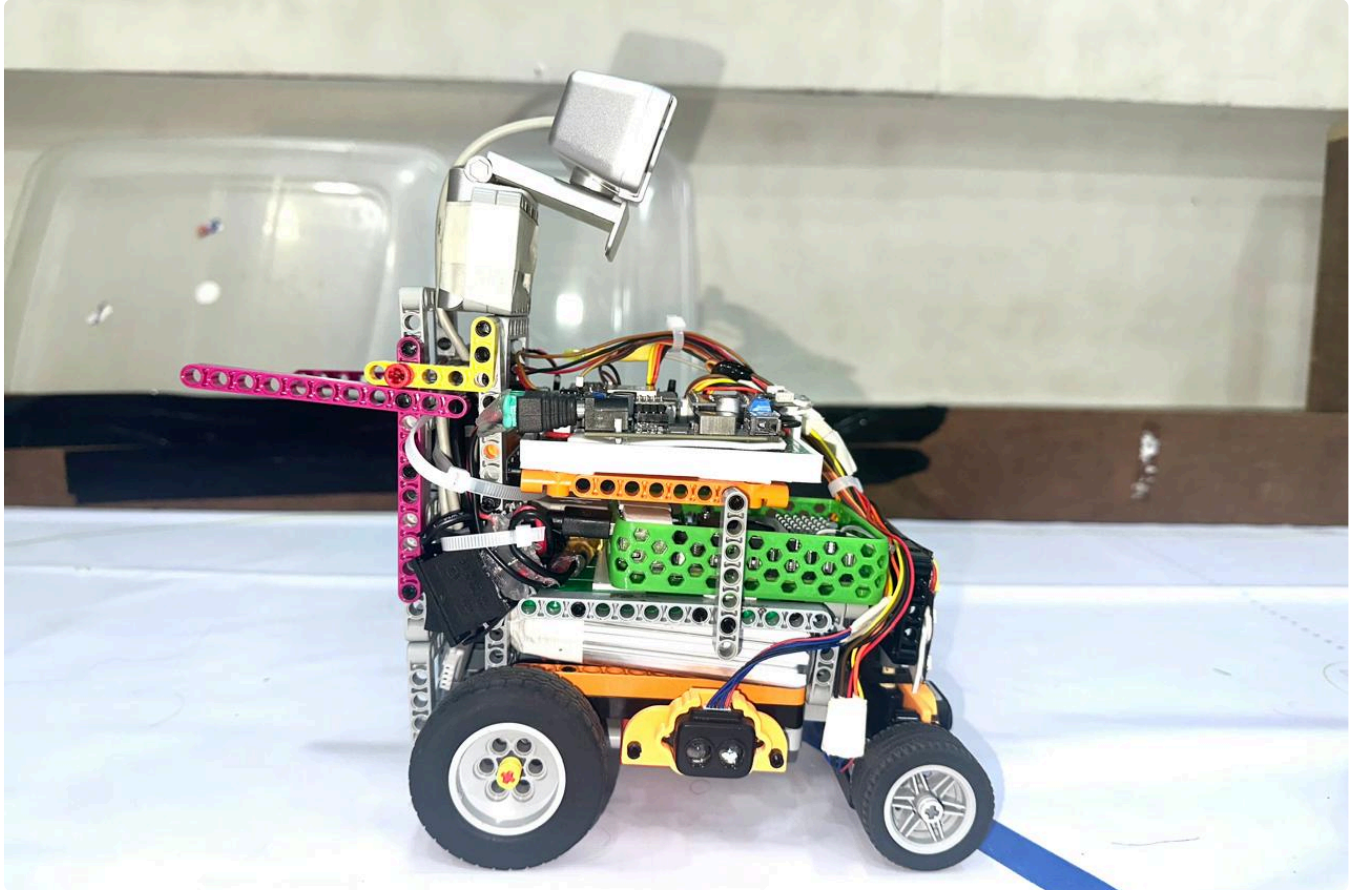


WRO FUTURE ENGINEERS 2026 · APAC

Engineering Journal

APAC 2026 edition (rev 5) · Team TED Drive



Team	Ethan Fernandes (software) · Tejas Sirikonda (mechanical) · Deeyan Mehta (electronics)
Coach	Amey Chavan (OMOTEC)
Competition	WRO Future Engineers 2026, APAC, Hyderabad, September 2026
Repository	github.com/teddriveomo/wro-2026-future-engineers (release 0.5.0 and later)
Prepared	23 September 2026, against WRO 2026 General Rules, Appendix C

This journal describes the vehicle rebuilt for APAC, shown above, and is generated from the repository's own documents, so the journal and the repository say the same thing. The India Nationals journal (rev 3, 24 August 2026), which describes the first vehicle, is a separate file in the repository: `docs/engineering_journal_final.pdf`.

01 Snapshot

The APAC vehicle in one page; every figure is from this journal and the repository.

Vehicle	Rebuilt in September 2026: a new 84-step LEGO Technic frame (332 g bare), a steering column that turns the whole front axle 60° each way, the camera on a 24.8 cm tower pointing 12° down, six WS2812 LEDs on the front frame. The horizontal side rollers of the rebuild were removed on 22 September.
Mass	862 g race-ready with the Raspberry Pi 5 (measured 16 September, before the rollers came off).
Drive	MEX 6 V motor through a TB6612FNG driver to the rear wheels: 422 rpm free-running at the wheel and 0.89 m/s average over 3 m from a standing start (measured 21 September).
Power	3S 2200 mAh pack: 1.34 A idle, 1.93–2.02 A driving, 2.77 A worst case (each branch measured, 21 September).
Compute	Raspberry Pi 5 (camera, detection, decisions) and ESP32 (drive motor, servo, three TF-Luna, LEDs), linked by USB serial.
Software	<code>main.py</code> (the coach's working copy, 23 September): Lab-colour pillar detection with tape-line and black-wall corner cues, a state machine that recentres between the walls after every pillar and corner and steers off a side wall during pillar passes, two pillars per straight, and a hand-over to a parking module after the twelfth corner (<code>partial_parking.py</code> as committed; <code>parallel_parking.py</code> is one switch away).
Open Challenge	<code>open-challenge.ino</code> on the ESP32 alone: direction detection, wall following at 30 cm with heading hold, twelve 90° turns on the BNO055 heading, and a stop 150 cm from the wall ahead (§04).
Evidence	Bench measurements of drive, power and camera geometry (§02, §03); detection shown on the mat (§04); code stored byte-for-byte with MD5 checksums (§06).

Contents

§	Section	Appendix C
01	Snapshot	
02	Mobility: frame, drive, steering, camera placement, six views	C1
03	Power and sensing: LEDs, measured power budget, failure points, ESP32 pins	C2
04	Software and obstacle strategy: control loop, states, recovery, detection, parking, validation	C3
05	Engineering decisions: every change with its reason, evidence and cost	C4
06	Reproducibility: files, serial link, settings, provenance and checksums	C5

02 Mobility

How the car is built and moves (Appendix C, criterion 1).

Status 2026-09-22. After the WRO India National Championship (26–28 Aug 2026) the vehicle was rebuilt for APAC 2026. This page records what changed and why. Every figure is marked by its basis: *measured* on the vehicle, *team* figure not yet measured, *derived* by calculation from named inputs, or *vendor* specification. Documents 1–5 remain the record of the Nationals configuration.

1. Specification

	Nationals (Aug 2026)	APAC 2026 (Sep 2026)	Basis
Length × width × height	length 23.0 cm	19.1 × 14.3 × 27.2 cm (±0.5 cm)	measured
Mass	785 g	862 g race-ready, Raspberry Pi 5 included (2026-09-16); not re-weighed after the side rollers were removed (§7)	measured
Axle loads	—	front 201 g, rear 661 g (23 % / 77 %) — see §2	measured
Chassis	LEGO Technic space-frame, 69-step build	LEGO Technic frame, 84-step build (instructions); 332 g bare	build file; measured
Printed parts	TF-Luna holders, servo horn beam, motor clamp (models/)	the same STL files	team
Steering	single-servo Ackermann (MG90S)	non-Ackermann: an MG90 servo drives a vertical steering column that turns the front axle, 60° each side	team
Wheelbase	—	13 cm	team
Wheels	front 41 mm, rear 55.6 mm	front 45 mm, rear 55 mm (rear tyres chosen for grip). Two 30 mm horizontal side rollers were fitted in the rebuild (build steps 71–76) and have since been removed — see §7	team
Drive motor	N20 gearmotor, 12 V, 600 rpm, via TB6612FNG	MEX motor (Avishkaar), rated 6 V, 400 rpm, via TB6612FNG — see §3	team
Gearing	20T pinion → 28T crown on a LEGO differential (5:7)	unchanged	team
Battery	3S LiPo, 11.1 V, 2200 mAh	unchanged	team
Power	pack → fast-charge module → Pi 5; pack → Buck-2 → 5 V rail (TF-Lunas, servo); TB6612FNG from the pack (2 — Power & Sensors)	unchanged, plus the WS2812 LEDs on the 5 V rail; power switch on the battery lead; start button on the rear beams	team
Sensors	3 × TF-Luna on a PCA9548A, BNO055, USB camera on a front arm at ~10 cm	same sensors; the camera (Lenovo 300 FHD) now sits on top of a rear tower — see §4	photos; measured
Lighting	none	6 × WS2812 LEDs (three 2-LED strips) on the front frame — see §5	team

Wiring: [schemes/wiring_apac_2026.md](#). Photos: [v-photos/apac-2026-09-22/](#) — six views, 2026-09-22.

2. Weight and balance

Quantity	Value	Basis
Total, race-ready	862 g	measured
Front / rear axle	201 g / 661 g (23 % / 77 %); the two sum to the total	measured
Centre of mass	10.0 cm behind the front axle (13 cm × 661 / 862)	derived
Bare chassis	332 g, split 31 % / 69 %	measured

Quantity	Value	Basis
Everything added to the chassis	530 g, about 82 % on the rear axle	derived

Assuming the added parts share the chassis's split would have put 67 g too much on the front, which is why the full car was weighed per axle.

The driven rear wheels carry 77 % of the weight, which helps traction. The steered front wheels carry 23 %, on the smaller tyres, so front grip is what limits cornering. Planned check: measure the turning circle at walking pace and at race speed; a much wider circle at speed means the front is sliding.

3. Drive

- **Motor:** MEX motor (Avishkaar), rated 6 V and 400 rpm no-load (team).
- **Speed at the rated voltage:** $400 \text{ rpm} \times 20/28 = 286 \text{ rpm}$ at the rear wheels; $\times \pi \times 55 \text{ mm} = \mathbf{0.82 \text{ m/s no-load}}$ (derived).
- **Voltage the motor sees:** the TB6612FNG switches the pack (11.1 V nominal, 12.6 V full) with PWM, so the motor's average voltage is duty $\times$ pack voltage. With the Nationals firmware, full commanded speed was 100 % duty (12.47 V measured at the motor terminals) and half speed was 48 % duty (5.95 V) — see 2a. On a 6 V motor, duty above about 48 % on a full pack averages more than its rating. The APAC firmware writes the duty directly with an 8-bit `analogWrite` and no cap: `SPEED = 210` is 82 % duty, 9.1 V average on the nominal pack and 10.4 V on a full one, 1.5–1.7 times the motor's 6 V rating (derived). Turning (100, 39 %), parking (70, 27 %) and the collision escape (80, 31 %) stay under the rating. The motor temperature after a full 3-minute run has not been measured yet.
- **Measured 2026-09-21** (pack at 12.3 V, `SPEED = 210`, so 10.1 V average; ammeter in the motor wire unless stated):

Test	Result	Basis
Rear wheels lifted, free-running	0.33 A; rear wheel 422 rpm (slow-motion video of a taped wheel, counted over one minute) = 1.22 m/s	measured
Driving on the mat	0.599 A	measured
Wheels held (stall), ammeter on the battery lead	1.24 A, which is 1.51 A through the motor at 82 % duty	measured; motor figure derived
3 m straight line from a standing start	3.36 s, 0.89 m/s on average with the launch included	measured

- **Speed check.** Scaling the rated 400 rpm linearly to 10.1 V predicts 482 rpm at the rear wheel. The measured 422 rpm is 12.5 % lower, because linear scaling ignores the voltage lost in the winding and the drivetrain friction that shows up as the 0.33 A free-running current.
- **Motor model** (derived). Treating the motor as a standard DC motor (voltage = current $\times$ resistance + speed $\times$ motor constant) and fitting it to the free-running and stall points gives 6.71Ω including the driver and $0.128 \text{ V}\cdot\text{s}/\text{rad}$ at the gearmotor output. At the measured 0.599 A on the mat it predicts 0.94 m/s, and 3.37 s for the 3 m standing start (launch time constant 0.17 s). The measured time is 3.36 s.
- **Torque and grip** (derived; drivetrain efficiency beyond the free-running losses assumed 0.8). The rear axle gets about 0.04 N·m while cruising (1.4 N at the tyres, the rolling and drivetrain load on the mat) and 0.17 N·m at stall (6.2 N at the tyres). The rear tyres carry 6.5 N (661 g), so 6.2 N would need a friction coefficient of 0.95: a hard launch or a push against a wall spins the rear wheels on most surfaces before the motor stalls.

4. Camera placement

- **Mount:** lens centre $24.8 \pm 0.5 \text{ cm}$ above the floor; optical axis 12° below horizontal (78° from straight down); the hinge is glued at that angle (measured).
- **Field of view:** Lenovo lists a 95° diagonal field of view (product page), which is about 56° vertical and 87° horizontal at 16:9 (derived; not yet measured on this camera).

Quantity	Formula	Value
Where the optical axis meets the floor	$h / \tan \alpha$	117 cm ahead of the lens; about ± 10 cm per 1° of angle error
Nearest floor in view	$h / \tan(\alpha + \text{VFOV}/2)$	~ 29 cm from the lens; the lens sits ~ 10 cm behind the front wheels, so ~ 19 cm ahead of them
Separation of pillar bases at 0.5 m and 1.0 m	$\text{atan}(h / 0.5 \text{ m}) - \text{atan}(h / 1.0 \text{ m})$	12.5° (5.6° with the Nationals 10 cm mount)
Tops of walls 1–3 m away	$\alpha - \text{atan}((h - 10 \text{ cm}) / D)$	3.6° – 9.2° above the image centre

h = lens height, α = pitch below horizontal, VFOV = vertical field of view, D = distance to the wall.

Why the camera went up. Reading range from the image depends on how low a pillar's base sits (the Nationals rule, 2 — Power & Sensors §3). Raising the lens from ~ 10 cm to 24.8 cm makes that signal about 2.2 times stronger (12.5° against 5.6° between pillars at 0.5 m and 1.0 m), and the centre of the image now lands ~ 1.2 m ahead instead of 0.33–0.57 m. The separation keeps growing with height up to $h = \sqrt{Z_{\text{near}} \cdot Z_{\text{far}}}$, 55 cm for pillars at 0.3 m and 1.0 m, so the earlier statement in §3 of that document, that a higher mount compresses it, was wrong; it is corrected there.

What it costs.

1. A blind zone: a pillar within ~ 19 cm of the front wheels has its base below the frame, so the nearest-pillar rule needs a tie-break for boxes cut off at the bottom edge.
2. A view over the 100 mm walls: off-field objects can appear between the wall tops and the horizon, so the pillar search has to stay below the black wall band, not a fixed image row.

Planned checks: a tape on the floor slid until it just enters the bottom of the image (measures the blind zone directly); a start-position reference frame compared before each round (the hinge is glued, but this catches knocks).

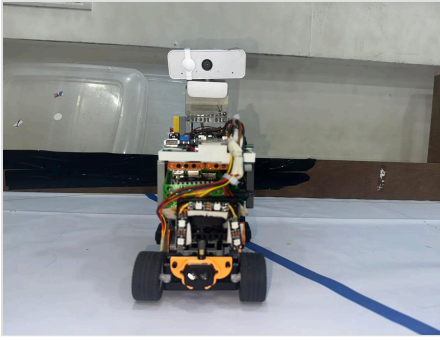
Colour calibration

The colour thresholds are set on the competition mat during one practice round at each venue, under that venue's lighting (team). Two tools in <src/apac-2026/tools/> apply the same image processing as `main.py`: a contrast and brightness scale, a gamma curve, then contrast-limited histogram equalisation (CLAHE, clip limit 2.0, 8×8 tiles) on the lightness channel.

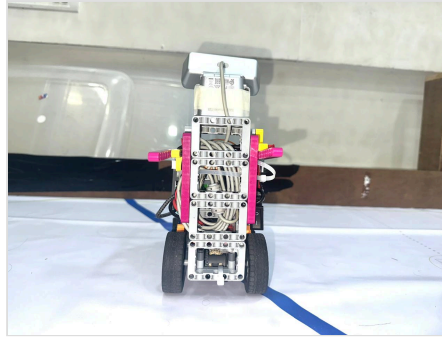
1. **Image.** `camera-settings.py` shows the live camera image with brightness, contrast and gamma sliders. The values chosen go into `BRIGHTNESS`, `CONTRAST` and `GAMMA` in `main.py` (committed: 0, 3.0 and 0.7).
2. **Colours.** `lab-calibration.py` converts the processed image to Lab and shows a mask while the lower and upper L, a and b sliders move. For each colour the sliders are set until only that colour shows in the mask, and the six values go into `COLOR_RANGES` in `main.py`.

Lab keeps lightness (L) apart from the two colour axes, so red (high a) and green (low a) can be separated with L left fully open, and a brighter or dimmer hall mostly changes L. The committed ranges do this: red is $a \geq 184$ and $b \geq 108$, green is $a \leq 101$, both with L from 0 to 255. The blue and orange tape lines are found with fixed HSV ranges (`COURSE_LINE_HSV_RANGES` in `main.py`), not with these tools.

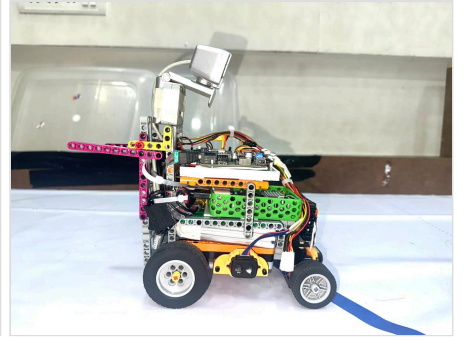
The six views



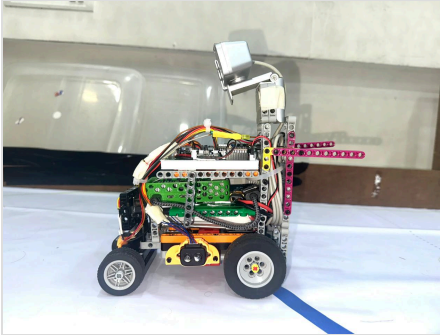
Front



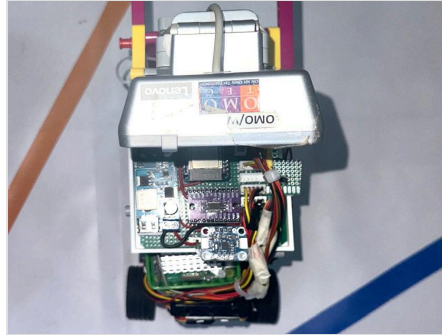
Rear



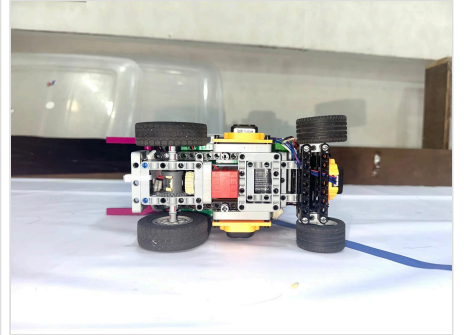
Left



Right



Top



Bottom (laid on its side)

03 Power and sensing

What powers the car, what it senses with, and what can fail (criterion 2).

5. Lighting and power additions

- **LEDs:** 6 × WS2812 on the front frame; data from ESP32 GPIO4; power from the 5 V Buck-2 rail (team). The firmware lights them white at `setBrightness(80)`, about 31 % (the bench measurements were taken at 50, about 20 %): roughly 19 mA per LED and 0.11 A for all six by the 60 mA-per-LED full-white rule of thumb in the [Adafruit NeoPixel Überguide](#) (derived). Full white would be 0.36 A.
- **Switch and button:** one power switch on the battery lead; the start button on the rear beams (team). The rules require exactly one power switch and one start button ([2 — Power & Sensors §6](#)).

Power budget, measured 2026-09-21

Each branch was measured on its own lead from the pack (ammeter and voltmeter, pack at 12.3 V). Pack currents for the two 5 V converters are given at the nominal 11.1 V, the conservative case used in [2a](#), with the converters assumed 88 % efficient; the motor branch is as measured.

Branch	Feeds	Basis	Idle	Driving	Worst case
Buck-2 (5 V)	3 TF-Luna, servo, 6 LEDs	measured at its input	0.13 A	0.23–0.32 A (servo sweeping)	0.32 A
Fast-charge module (5 V)	Pi 5, camera, ESP32	measured peak, 11.8 W at the Pi	1.21 A	1.21 A	1.21 A
TB6612FNG	drive motor	measured	0	0.49 A (0.599 A × 82 % duty)	1.24 A (stall)
Pack total			1.34 A	1.93–2.02 A	2.77 A

With the Pi's peak treated as its average, a 2200 mAh pack used to 80 % gives about 52 minutes of driving, about 17 three-minute runs (derived). The prediction for the Nationals car in [2a](#) was 0.95–1.40 A driving; the difference is the Pi (an 11.8 W peak against 6.2–8.5 W predicted as an average) and the larger motor.

Part	Load	Rating (vendor)	Use
TB6612FNG, one channel	1.51 A through the bridge at stall; 0.60 A driving	1.2 A continuous, 3.2 A peak	126 % at stall ; 50 % driving
Pi 5 USB ports without USB-PD	camera + ESP32	600 mA in total	see §6
Battery, 3S 2200 mAh 60C	2.77 A worst case	132 A	2 %

6. New failure points

Risk	Effect	Check or mitigation
Pi 5 fed from a USB-A fast-charge module (no USB-PD)	The Pi peaked at 11.8 W (2.36 A at 5 V, measured) and the module's rating has not been read; without USB-PD the Pi limits its USB ports to 600 mA in total, and the camera and the ESP32 both draw from them	Run <code>vcgencmd get_throttled</code> after a full run; move to a 5 A USB-C PD supply if it reports problems
WS2812 data at the ESP32's 3.3 V	The LED input-high threshold is $0.7 \times V_{DD}$ (3.5 V at 5 V); a marginal signal can flicker or show the wrong colour, tinting the scene	Watch for flicker in the camera image; add a level shifter if seen
6 V motor on the 11.1 V pack	Heat and brush wear: <code>SPEED = 215</code> averages 9.4–10.6 V, above the 6 V rating (§3)	Temperature check after a 3-minute run

Risk	Effect	Check or mitigation
Drive motor held against a wall	1.51 A through the TB6612FNG at stall, above its 1.2 A continuous rating (§5); the driver's thermal shutdown or the motor's heating would stop the car	The Pi's distance logic backs away (escape at 31 % duty for 0.30 s, at least 3 cm of progress, at most 2 attempts); on most surfaces the rear wheels spin before a full stall (§3)
Firmware front stop: the ESP32 stopped the drive motor when the front TF-Luna read under 18 cm or its reading went stale	Observed: it stopped the car and ended a run (team)	Removed from the firmware on 2026-09-10; collision handling moved to the Pi (§7)
Camera angle	Pixel-based settings shift if the angle moves	Hinge glued; reference frame before each round
Background above the walls	Off-field red or green objects read as pillars	Search only below the wall band
Blind zone ahead of the wheels	The nearest-pillar rule ties for boxes cut off at the bottom	Tie-break rule in the code

8. ESP32 pin map

From the Nationals firmware ([round2_ino.ino](#)) and the 2026-09-20 wiring draft; only GPIO4 is new. Checked against the APAC firmware ([src/apac-2026/](#)) on 2026-09-21: it reads the centre TF-Luna on mux channel 3 (channel 1 at Nationals).

Signal	GPIO
I ² C SDA / SCL — PCA9548A at 0x70: ch0 left, ch3 centre, ch2 right TF-Luna (0x10), ch4 BNO055 (0x28)	21 / 22
Steering servo	13
TB6612FNG AIN1 / AIN2 / PWMA / STBY	25 / 26 / 33 / 27
WS2812 data	4
Start button	32

04 Software and obstacle strategy

How main.py drives the Obstacle Challenge and how the car parks (criterion 3).

Scope. How the Obstacle Challenge code works. §1-§6 describe [main.py](#) as released in v0.4.0 (from the coach's commit of 2026-09-08; [that file](#)); §10 lists what the current version, from the coach's working copy of 23 September 2026, changes. §8 covers the parking modules, §9 the pillar and line detection of the current file and §11 the Open Challenge firmware. Names in capitals are states; names in `code` are constants, given with their values. The Nationals software is in [3 - Software](#) and the vehicle in [APAC 2026 vehicle](#).

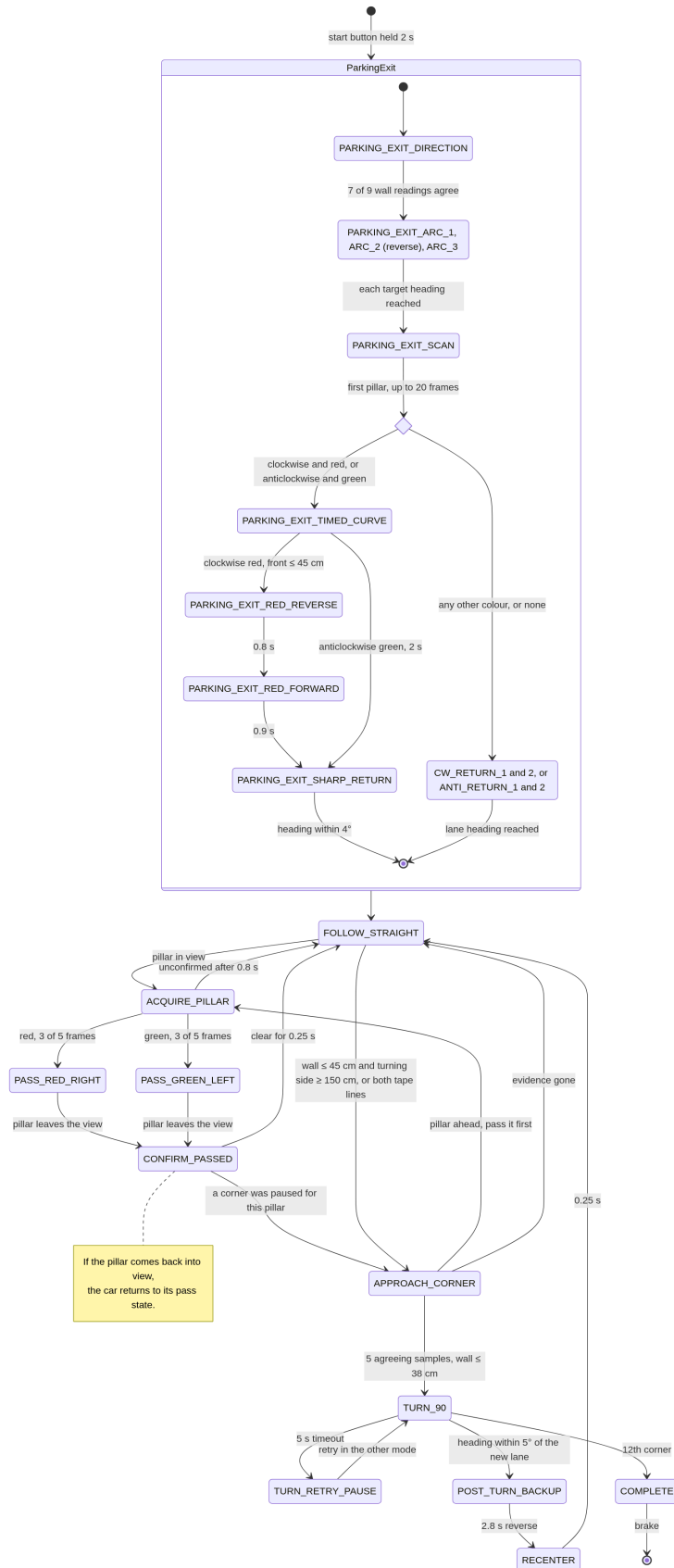
1. The control loop

In a loop that sleeps 10 ms between passes (`NAVIGATION_LOOP_SECONDS`), the Raspberry Pi:

1. reads the newest ESP32 telemetry line: heading, the left, centre and right TF-Luna distances, and a validity mask;
2. takes the newest camera frame and runs the pillar and tape-line detector (`UnifiedPillarDetector`, §9);
3. calls the controller (`ObstacleChallengeController.update`), which sends one `speed,direction,steer` command to the ESP32.

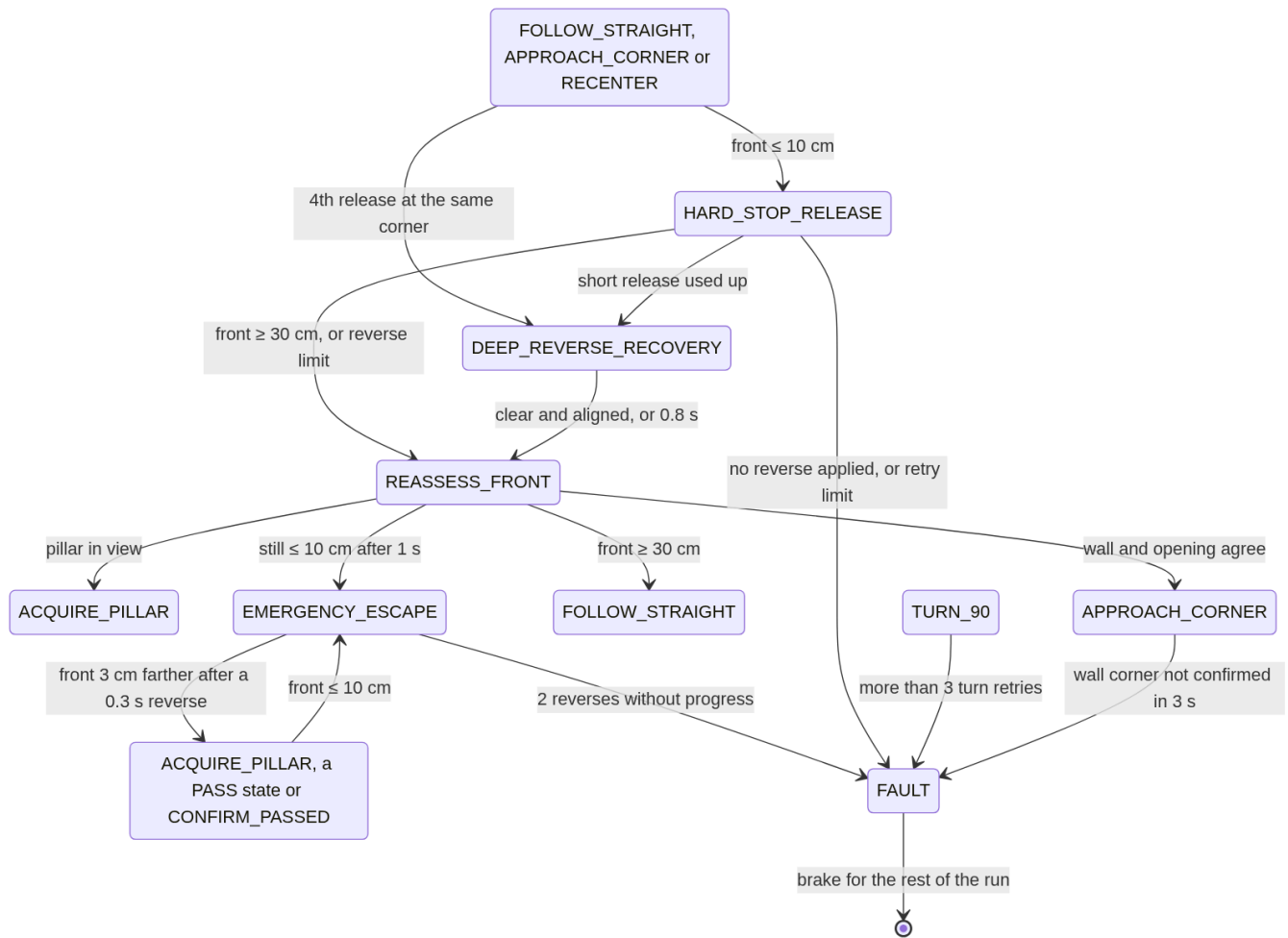
The run starts once the start button has been held for 2 s: the ESP32 then sends `OK` and the Pi answers `OK`. It ends in `COMPLETE` after 12 corners (`COUNTER_MAX`), that is three laps. Parking exit is on (`ENABLE_PARKING_EXIT = True`); parking at the end is off (`ENABLE_PARKING_IN = False`). Every state change is printed as `[NAV] OLD -> NEW: reason`.

2. The run



- Red pillars are passed on the right and green ones on the left.
- From any pillar state, a pillar seen beyond both tape lines within 5 s of a pass (`ALPHA_POSITION_WINDOW_SECONDS`) belongs to the next straight, so the car goes to `APPROACH_CORNER` and takes the corner first.
- Every driving state brakes and holds while the telemetry is older than 0.30 s (`TELEMETRY_STALE_SECONDS`), a TF-Luna or the heading is invalid, or the camera frame is older than 0.30 s (`CAMERA_STALE_SECONDS`). It resumes in the same state, and the hold does not count against that state's timers.

3. Recovery and faults



EMERGENCY_ESCAPE resumes whichever state it interrupted. The parking-exit states also go to FAULT when a timed step overruns (4 s to reach the first red pillar, 5 s to reach the lane heading).

4. Why the states are split this way

State	Reason
PARKING_EXIT_DIRECTION	The lane direction must be known before the first corner. The code takes it from which side wall is nearer: 7 of the last 9 readings must agree, each with left and right at least 15 cm apart (DIRECTION_MIN_DIFFERENCE_CM).
PARKING_EXIT_SCAN	A pillar may stand just outside the parking lot. If it is on the side the car turns toward (red when driving clockwise, green anticlockwise), a timed pass keeps the car on its correct side; otherwise two heading arcs return the car to its lane.
ACQUIRE_PILLAR	One frame of red or green must not steer the car, so a pillar is followed only once 3 of 5 frames confirm it.
CONFIRM_PASSED	The camera loses a pillar before the rear of the car has cleared it, so the car holds its line for 0.25 s (PILLAR_CLEARANCE_HOLD_SECONDS).
APPROACH_CORNER	Neither a tape line nor a short front distance alone proves a corner. A turn needs the end wall within 38 cm (CORNER_TRIGGER_CM), the side the car turns toward open to at least 150 cm (OBSTACLE_CORNER_SIDE_OPEN_CM) and the heading within 8° of the straight, in 5 agreeing samples over at least 0.30 s.
TURN_90	The target is the straight's heading plus or minus 90°, reached within 5° (TURN_HEADING_TOLERANCE). When the outer side has at least 60 cm of room (CORNER_REVERSE_CLEARANCE_CM), the turn starts with a reverse arc of up to 0.8 s; otherwise it is driven forward.
POST_TURN_BACKUP	After each corner the car reverses for 2.8 s (POST_TURN_BACKUP_SECONDS) so that the next straight and its pillars fit in the camera's view.

5. Edge cases the code handles

Situation	What happens
Telemetry or camera stale, or a sensor invalid	Brake and hold the current state; resume when fresh.
The same pillar seen again just after a pass	Ignored for 0.6 s (<code>PILLAR_REACQUIRE_COOLDOWN_SECONDS</code>).
Candidate pillar not confirmed	Back to FOLLOW_STRAIGHT after 0.8 s (<code>PILLAR_ACQUIRE_TIMEOUT_SECONDS</code>).
Pillar ahead during a corner approach	The corner is paused, the pillar is passed, then the corner approach resumes.
Pillar beyond the corner seen right after a pass	The corner is taken first (§2).
Front within 10 cm	A short reverse, then a fresh check for a pillar, a corner or a clear front before moving on (§3).
Repeated releases at one corner	After the third, a longer reverse at speed 55 for up to 0.8 s, aiming for 70 cm of clearance.
Turn not finished in 5 s	Retry with the other turn mode (forward or reverse), up to 3 retries.
12th corner	COMPLETE: the car brakes and stays stopped.

6. Known limits

- The driving direction is locked at the start and does not change during the run.
- The last-resort front trigger (`PYTHON_EMERGENCY_RELEASE_CM = 10`) is inside the TF-Luna's blind zone: its specified range starts at 0.2 m (vendor). Corners are normally handled from 45 cm, so this trigger only acts after a misjudgement.
- Since the firmware stop was removed on 2026-09-10, the ESP32 always reports `hardStop = 0`, so the `hard_stop` checks in `main.py` never fire; some comments in `main.py` still describe an ESP32 stop at 18 cm.
- With parking at the end off, `PARKING_ENTRY_SEARCH` and the `PARKING_IN_` states are never reached. No transition leads into the `PARKING_EXIT_CW_RED_` and `PARKING_EXIT_ANTI_GREEN_` states.
- `FAULT` stops the car for the rest of the run.
- After the twelfth corner `main.py` hands over to the parking module its switches select; `main.py` stops with an error if both or neither are on (§8).
- Both parking modules end the entry when the front TF-Luna reads 5 cm or less (`ENTRY_STOP_FRONT_CM = 5`), and `partial_parking.py` also uses limits of 6, 8 and 15 cm (`REVERSE_BLOCK_SEARCH_SIDE_STOP_CM`, `REPOSITION_SIDE_CLEARANCE_CM`, `REPOSITION_FRONT_CLEARANCE_CM`). All are inside the TF-Luna's blind zone: its specified range starts at 0.2 m (vendor).
- In `parallel_parking.py`, `FINAL_REVERSE`, `FINAL_BRAKE` and `FINAL_FORWARD` are defined, but no transition leads into `FINAL_REVERSE`, so none of the three is reached; `PARALLEL_ALIGN` ends the parking in `COMPLETE`.
- The docstrings of `parallel_parking.py` say its detector does not use the blocks' colour; the code (`ParkingGeometryDetector.extract`) keeps only magenta pixels.

7. Validation

The metrics that exist, and how the next ones are collected.

- **Detector, per frame.** The live view shows the tracked pillar's confidence, hits out of 5, bottom position and area band (§9); in the two captured frames of 2026-09-12 the track had 5 of 5 hits at confidence 0.92 and 0.86.
- **Nationals stack.** The test levels T1-T7 in [tests.md](#) (data integrity, model evaluation, operating-point sweep, deploy parity, on-target latency, firmware bench, track), with what was run and what was not, and the track recordings in `other/test-runs-2026-08-08` and `other/test-runs-2026-08-09`.
- **APAC bench, 2026-09-21.** Drive and power measurements in [vehicle §3](#) and [§5](#): free-running and stall currents, 422 rpm at the rear wheel, 0.89 m/s over 3 m from a standing start, the current of each supply branch.
- **Run logs.** `main.py` can log every run: with `ENABLE_RUN_CSV_LOGGING = True` (committed as `False`) it writes `run_logs/round2_run-<date_time>.csv` beside itself, one row per telemetry sample and one per event, with lap, corner count, pillars this straight, state, heading and target, the three distances, the command sent and applied, the tracked pillar with its band, and the tape depths. Those files give corners completed, pillars passed,

recoveries and faults per run; none is in this repository yet. For parking, `parallel_parking.py` with `ENABLE_RUN_LOGGING = True` (committed as `False`) writes a CSV and an annotated video per attempt (§8).

8. Parking modules

Two files beside `main.py` park the car at the end of the run. After the twelfth corner `main.py` enters `PARKING_PARALLEL_HANDOFF` and calls the one its switches select (`PARTIAL_PARKING = True` and `PARALLEL_PARKING = False` as committed; exactly one must be on). Each can also run on its own, and each imports `main.py` for the camera, the serial link, telemetry, the live view and the shutdown. At the hand-over `main.py` first registers itself as the module `main` (`sys.modules`), so the parking module reuses the camera, serial port and telemetry already running ("give the standalone parking module the same initialized camera, serial port, telemetry, and safety API"). Constants named in this section are in the parking files.

File	What it does	Start
<code>parallel_parking.py</code>	Parallel parking (<code>BUILD_ID = "slot-guided-entry-v29"</code>): finds the two parking-lot limitations with the camera, drives into the gap between them and straightens up	<code>python parallel_parking.py --direction clockwise --last-pillar auto</code>
<code>partial_parking.py</code>	Direct entry: the same approach and block search, then a drive into the space on a locked heading ("Standalone 90 degree parking" in its help text)	<code>python partial_parking.py --direction clockwise --last-pillar auto</code>

`--direction` is `clockwise` or `anticlockwise`; `--last-pillar` is `auto` (detect a red or green final pillar), `red`, `green` or `none`. Both wait for the start signal as `main.py` does, and end in `COMPLETE` with the brake held (exit code 0) or in `FAULT` with a printed reason (exit code 1).

Which one. A full parallel park scores 15 points against 7 for a partial or non-parallel one (2026 General Rules, scoring element 1.8.2), and touching a limitation ends the round (rule 9.24.7). As committed, the switches select `partial_parking.py`; parking parallel means setting `PARALLEL_PARKING = True` and `PARTIAL_PARKING = False`. Every `main.py` name the parking modules use exists in the committed `main.py`.

Camera calibration. Both files need `parking_camera_calibration.json` beside them. It depends on how the camera is mounted; the team's file, made with the September camera mount, is committed beside them. To make a new one: `python parallel_parking.py --calibrate` shows the camera image, and clicking the four corners of a rectangle on the floor (near-left, near-right, far-right, far-left; 100 × 150 cm unless `--calibration-width-cm` and `--calibration-depth-cm` say otherwise) saves the homography from image to floor. If the file is missing, `parallel_parking.py` starts this calibration itself and exits after saving.

Finding the space (`parallel_parking.py`)

- Colour mask.** Pixels that are magenta in both HSV (`MAGENTA_HSV_LOW` 135, 65, 45 to `MAGENTA_HSV_HIGH` 179, 255, 255) and Lab ($a \geq 145$, $b \leq 175$). The top 28 % of the image is ignored (`ROI_TOP_FRACTION = 0.28`); one opening and two closings with a 5 × 5 kernel remove specks and fill gaps.
- Shape.** Each contour's rotated rectangle must fit a 200 × 20 × 100 mm limitation seen face-on, edge-on or at an angle: aspect 1.15 to 6, rectangularity at least 0.42, magenta fill at least 0.45, width at most 30 % and height 5.5 to 72 % of the image, lower edge below 55 % of the image height and not cut off by the bottom of the image.
- Floor position.** The midpoint of the rectangle's two lowest corners goes through the calibration homography, which gives the block's position on the floor in cm.
- Two blocks.** A block is confirmed after 3 hits in its last 6 frames (`BOUNDARY_REQUIRED_HITS`, `BOUNDARY_HISTORY`). Block 1 is the best-scoring rectangle on the parking side of the image. Block 2 must not overlap it, and must be 25 to 120 cm from it on the floor, within 55 cm of it sideways and at least 15 cm farther away.
- Slot.** The slot is the floor midpoint of the two blocks. It is confident when both blocks are confirmed, neither has been missed for more than 2 frames, they are 25 to 120 cm apart and both have a confidence of at least 0.55. Its bearing from the camera steers the car (`SLOT_BEARING_KP = 2.0`) and is used only within $\pm 30^\circ$ (`SLOT_MAX_USABLE_BEARING_DEG`).

States (parallel_parking.py)

State	What the car does	Leaves when
SEARCH_LAST_PILLAR	Looks for the final pillar with <code>main.py</code> 's pillar detector	a pillar is confirmed: <code>PASS_LAST_PILLAR</code> ; no pillar and block 1 confirmed: <code>TRACK_OPENING</code>
PASS_LAST_PILLAR	Steers past the final pillar	red path: <code>SEARCH_FIRST_BOUNDARY</code> ; green path: <code>GREEN_RETURN_STRAIGHT</code> back to the start heading, <code>GREEN_FORWARD_STRAIGHT</code> for 1.0 s, then <code>SEARCH_FIRST_BOUNDARY</code>
SEARCH_FIRST_BOUNDARY	Drives at speed 50 on the start heading turned 45° towards the parking side (5° after a green-path pillar). If block 1 is not confirmed within 4 s, a short reverse (0.6 s at speed 60, steer 45) and the search restarts; each retry adds 5° of offset (up to 55°), 0.2 s of search time and a longer, sharper reverse (up to 1.0 s, steer 55; the speed stays 60)	block 1 confirmed: <code>TRACK_OPENING</code>
TRACK_OPENING	The same search	block 2 confirmed and the slot confident: <code>ALIGN_VIRTUAL_CUTOUT</code>
ALIGN_VIRTUAL_CUTOUT	Speed 50, steering on the slot bearing (at most 25)	8 consecutive updates with a usable bearing (<code>ALIGN_CONFIRM_SAMPLES</code>): <code>ENTER_SPACE</code> ; front TF-Luna at 5 cm or less: <code>PARALLEL_ALIGN</code>
ENTER_SPACE	Speed 70, steering on the slot bearing (at most 12)	front TF-Luna at 5 cm or less: <code>PARALLEL_ALIGN</code>
PARALLEL_ALIGN	Speed 50 at full correction steer (50): reverse strokes of 0.8 s, then forward until the front TF-Luna reads 5 cm	heading within 2° of 0° on 3 consecutive updates: <code>COMPLETE</code>

The red and green paths swap in an anticlockwise run (`pillar_behavior`). After a green-path pillar, each block search ends in `FAULT` after 15 s (`GREEN_PARKING_SEARCH_MAX_SECONDS`), and the return to the start heading after 10 s.

What `partial_parking.py` changes

- **No final pillar.** With `--last-pillar none`, or when no final pillar is confirmed within 3 s, it drives straight for 2 s at speed 80 (`NO_PILLAR_STRAIGHT`), then searches for block 1 in reverse at speed 50 (`REVERSE_BLOCK_SEARCH`).
- **Green path.** Clockwise with a green final pillar (anticlockwise with a red one) it steers 45° forward until the centre TF-Luna reads 30 cm, then turns slowly at full lock the other way to a set heading before the block search (`GREEN_CW_TURN_RIGHT`, `GREEN_CW_TURN_LEFT`; mirrored as `RED_ACW_TURN_LEFT`, `RED_ACW_TURN_RIGHT`).
- **Opening check.** Where the parallel module would start aligning, it requires a visible opening at least 6.5 % of the image wide (`MIN_VISIBLE_ENTRY_GAP_RATIO`) and a slot at least 20 cm ahead with a valid bearing. A narrower opening gets one repositioning (`REPOSITION_AWAY`, `REPOSITION_FORWARD`, `REPOSITION_RETURN`) before a `FAULT`.
- **Entry.** It locks the entry heading at the current heading plus the slot bearing (at most $\pm 30^\circ$) and drives in at speed 70, the parallel module's `ENTRY_SPEED` (`ENTER_SPACE`). Steering is $2.0 \times$ the heading error plus $0.6 \times$ its integral, which builds only once the slot is closer than 40 cm, plus a smoothed term of at most 8 that centres the opening in the image while the slot is 40 cm or more away; at most 24 in total.
- **Stop.** At a front reading of 5 cm or less, or when the front reading is missing or stale, it brakes (`FRONT_SENSOR_HOLD`): 3 fresh readings at the stop move on to `PARALLEL_ALIGN`, and 3 fresh clear readings resume the previous state. `PARALLEL_ALIGN` works as in the parallel module but counts only fresh telemetry.

Why it is built this way

Reasons as the code's comments give them.

Choice	Reason given in the code
The TF-Luna sensors are not used while searching for or aligning with the blocks	The blocks' geometry comes from the full camera view; distance matters only once the car is in the space
A short camera miss does not brake the car	The brake is not pulsed between camera frames: the selected motion continues until vision recovers, and during alignment the car drives straight and restarts the confirmation
Avoiding the final pillar starts on its first sighting, but the pass needs several frames of confirmation	At driving speed a one-frame delay can put the pillar past the useful steering window; a false detection can steer briefly but cannot latch the pass
The block search holds a heading offset towards the parking side	So that the blocks enter the camera view; an earlier path turned into the parking area before block 1 was confirmed
Reverse strokes in PARALLEL_ALIGN stop after 0.8 s; forward moves stop on the front TF-Luna	So that the car cannot back out of the space
The shape limits are broad	A limitation can be seen edge-on, face-on, at an angle or cut off by the image edge
One magenta contour never updates both blocks, and a sudden inward jump is taken as the next block	Block 1 leaves the image through its left edge in a clockwise run (the right edge anticlockwise), so a new rectangle further in is block 2, never a new sample of block 1
<code>partial_parking.py</code> ignores a pillar cut off by the image edge	Such an object belongs to the next lane, not to the final pillar ahead

Failure exits

Every FAULT prints its reason and holds the brake.

- `parallel_parking.py`: after a green-path pillar, the return to the start heading takes more than 10 s, or block 1 or block 2 is not found within 15 s.
- `partial_parking.py`: no valid IMU heading when the entry heading is locked; the opening is still narrower than 6.5 % of the image after one repositioning; the slot is not ahead with a valid bearing; something in front while the final pillar is classified, during the no-pillar straight or during the pillar pass; the pillar pass takes more than 4 s; too little side clearance in the reverse search; too little front or side clearance while repositioning; the opening is lost before the car passes it.

9. Pillar and line detection

What `UnifiedPillarDetector` does with each 1280 × 720 frame, and why. Constants are in `main.py` (the current file; v0.4.0 values in brackets where they differ); the two figures at the end show the result on the mat.

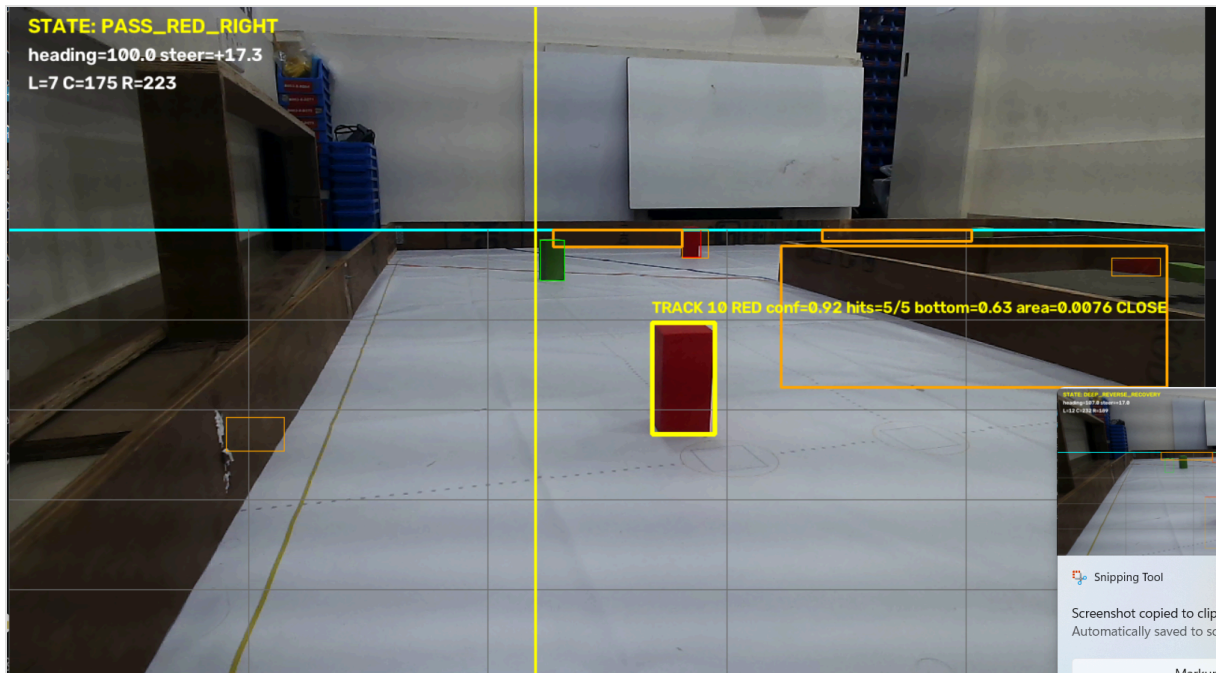
1. **Image preparation.** The frame is stretched (`CONTRAST = 3.0`, `BRIGHTNESS = 0`) and gamma-corrected (`GAMMA = 0.7`), then converted to Lab; CLAHE (clip 2.0, 8 × 8 tiles) evens out the lightness channel. Lab keeps lightness apart from the two colour axes, so red (high a) and green (low a) separate with L left open, and a brighter or darker hall mostly moves L ([vehicle §4](#)). The venue's thresholds come from one practice round with `lab_calibration.py`.
2. **Side masks.** Before detection a strip at each side of the frame is set to neutral grey (128, outside every Lab range) without moving pixel coordinates (`asymmetric_camera_view`): 16 % of the width on each side on a straight (`CAMERA_SIDE_MASK_FRACTION`). After a corner the two sides are masked separately, depending on the turn direction (`post_turn_view_fractions`, `POST_CORNER_VIEW_SIDE_FRACTION = 0.20`), until the new straight is established: 1.5 s of forward driving, or 12 s. Not in v0.4.0.
3. **Colour masks.** One Lab range per colour (`COLOR_RANGES`, as committed): red a at least 184 and b at least 108; green a at most 101; blue a 153 to 186 and b at most 90; orange a at least 130 and b at least 150; L unconstrained. Each mask is opened once and closed twice with a 3 × 3 kernel, and the top third of the image is cleared (`DETECTOR_ROI_TOP_RATIO`): from the 24.8 cm mount that third lies above the tops of the walls out to about 3 m ([vehicle §4](#)), so it holds only the hall.
4. **Candidate shapes.** Each contour is tested by size and shape. Pillars (red, green): area at least 0.045 % of the frame and height at least 2.5 % of the frame height; height to width 0.65 to 8 (a 50 × 50 × 100 mm sign is about twice as tall as wide; the range allows partial views and perspective); rectangularity at least 0.35 and solidity at

least 0.60, since a pillar is a solid rectangle and tape edges and reflections are not. Tape lines (blue, orange): area at least 0.03 %, height at least 1 %, height to width 0.2 to 10, rectangularity at least 0.25, solidity at least 0.50. Confidence = $0.20 \times \text{area} + 0.20 \times \text{height} + 0.25 \times \text{rectangularity} + 0.35 \times \text{solidity}$, with area and height capped at $5 \times$ the minimum area and 25 % of the frame height; candidates under 0.45 are dropped (`DETECTOR_MIN_CONFIDENCE`).

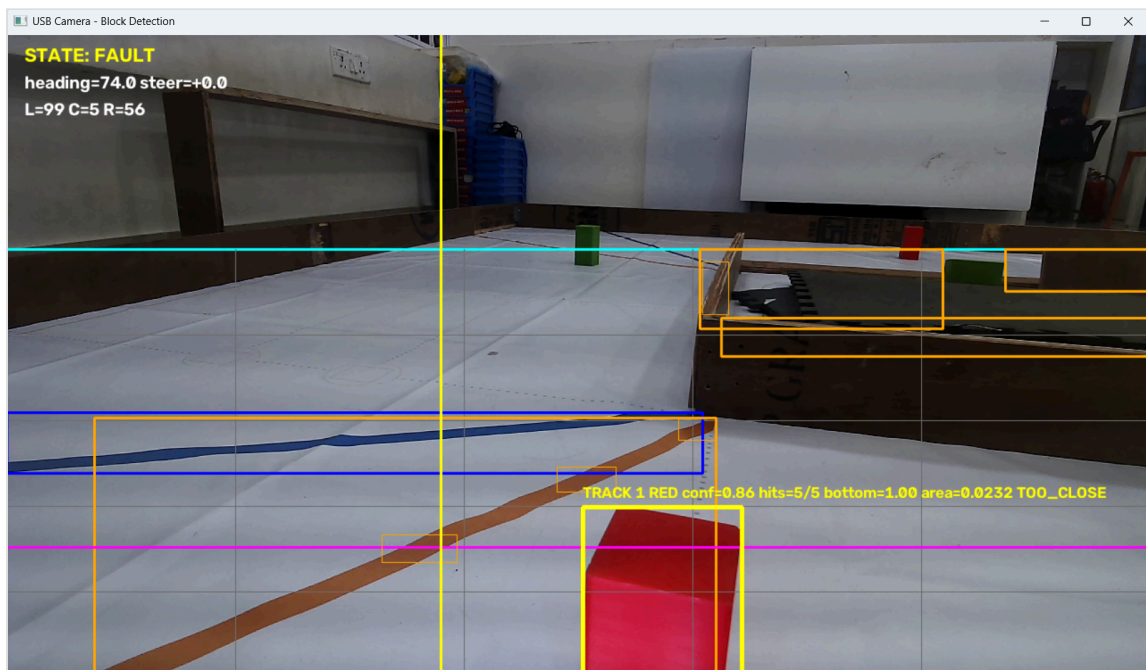
5. **Tape lines and the corner reference.** The mat's blue and orange lines mark the approach to a corner. They are found in HSV (blue hue 95-135, orange hue 3-22) below the top third, with the same morphology; a line must cover 0.025 % of the frame, span 10 % of its width and be at least twice as wide as tall. The lowest line's mid-height is its depth; when both colours are visible, the deeper of the two is the corner reference (the magenta line in the live view). The current file also confirms a corner from the black walls: in HSV (value at most 70), a broad black end wall with an opening on the turn side, confirmed in 3 of the last 4 frames (`BLACK_CORNER_*`).
6. **Which pillar counts.** With the corner reference visible (and not suppressed for 3 s after a turn, `POST_TURN_TAPE_IGNORE_GRACE_SECONDS`), a pillar whose centre is above the reference, farther away than the tape, is ignored when it is entirely above it or sits in an outer column of the 5×5 grid: it stands beyond the corner or in a side lane (`IGNORE_CORNER` in the live view). Two rules keep the car from dropping the obstacle in front of it: the track the controller is already passing keeps priority even when tape moves in front of it (`FRONT_PRIORITY`), and when a larger, lower interior pillar and a smaller outer pillar are both beyond the reference, the interior pillar stays actionable if it is $1.25 \times$ larger (`CORNER_FOREGROUND_AREA_ADVANTAGE`). A small ignored pillar never suppresses a valid foreground pillar.
7. **Tracking.** One pillar is tracked at a time. A new track starts on the best candidate by $0.55 \times \text{bottom} + 0.35 \times \text{confidence} + 0.10 \times \text{area}$: the lowest in the image is the nearest. Each frame the track is matched to a same-colour candidate by centre distance (at most 0.25 of the frame) or overlap (IoU at least 0.05), and its position is smoothed ($0.65 \text{ new}, 0.35 \text{ old}$). A track is confirmed after 3 hits in its last 5 frames (`DETECTOR_CONFIRMATION_HITS`, `DETECTOR_TRACK_HISTORY`); an unconfirmed track is dropped after 2 misses (`DETECTOR_UNCONFIRMED_MISSES`); a confirmed track stays locked until the controller records the pass, so a brief occlusion cannot switch the target. Confidence is $0.75 \times \text{the detection's confidence} + 0.25 \times \text{hits}/5$, and decays by 10 % per missed frame.
8. **Two pillars per straight** (current file). At most two pillars are taken on each straight (`MAX_PILLARS_PER_STRAIGHT` = 2); once both are passed, no new pillar is taken until the corner (live view: `CORNER NEXT - NO NEW PILLAR`). If no second pillar has come 3.6 s after the first, slot 2 is filled and the corner takes priority (`SECOND_PILLAR_SLOT_TIMEOUT_SECONDS`). One high track in the next lane is deferred rather than its whole colour suppressed, and right after a turn nearby pillars take priority (`defer_background_candidate`). A wide, low rectangle, at least twice as wide as tall, is treated as a parking-lot limitation, not a pillar; in laps 1 and 2 the car steers gently around it (speed 60, steering 15).
9. **Range and control.** The pillar's distance band comes from its area: far under 3,200 px×px (4,000), near under 7,000, close under 20,000, otherwise too close; the 1280×720 breakpoints are kept as ratios of the frame (`PILLAR_AREA_NEAR_RATIO`, `PILLAR_AREA_CLOSE_RATIO`, `PILLAR_AREA_TOO_CLOSE_RATIO`). Each band selects a speed, gain and steering limit (`PILLAR_DISTANCE_PROFILES`; far: speed 100, gain 0.70, 3 to 12 of steer (5 to 18); too close: speed 80, gain 1.50, 38 to 52). The controller steers the pillar to a target column, red to 38 % of the width (25 %) so the car passes on its right and green to 62 % so it passes on its left (`PILLAR_RED_TARGET_X`, `PILLAR_GREEN_TARGET_X`), with gain 200 (`PILLAR_LATERAL_KP`), at most 40 of correction and smoothing 0.65.

The live view (`annotate`): the cyan line is the top of the search area; the grey grid is the 5×5 columns and rows; blue and orange boxes are tape lines; the magenta line is the corner reference; thin boxes are candidates; a magenta box is a pillar ignored as beyond the corner (`IGNORE_CORNER Cn`, n = grid column); a yellow box marks the pillar being passed (`FRONT_PRIORITY`); the tracked pillar's box is yellow once confirmed and cyan before, labelled `TRACK id COLOUR conf hits/5 bottom area BAND`, with a vertical line at its target column; the text lines give the state, heading and steer, and the three TF-Luna readings in cm. The current file adds `PILLAR SLOTS n/2`, `CORNER NEXT - NO NEW PILLAR`, `SLOT 2 TIMEOUT IN n s`, `BLACK CORNER` and `VIEW MASK L=n% R=n%`.

Two frames captured during testing on 2026-09-12 with an earlier version: the overlay is that of v0.4.0's `annotate`, and the areas fall in v0.4.0's bands.



Passing a red pillar: state PASS_RED_RIGHT, steer +17.3 at heading 100.0; TF-Luna L 7, C 175, R 223 cm; track 18, red, confidence 0.92, 5 of 5 hits, bottom 0.63, area 0.0076, band close; orange tape boxes ahead and the red target column drawn.



A run that ended in FAULT: a red pillar 5 cm from the centre TF-Luna (L 99, C 5, R 56 cm), track 1 at confidence 0.86, bottom 1.00, area 0.0232, band too close; both tape lines are visible and the magenta corner reference is drawn. The fault reason is printed to the terminal, not to the view, and was not captured.

10. What the current main.py changes

The file now in the repository, from the coach's working copy of 23 September 2026, has 5,685 lines against 4,771 in v0.4.0. This section lists what changed, from the code; where the code gives a reason in a comment or docstring, it is quoted or paraphrased. The team removed the side rollers after the wall work ([vehicle §7](#)).

Walls. Four mechanisms keep the car off the walls:

- PILLAR_RECENTER follows every pillar pass. It counter-steers 22° against the pass for 0.18 to 0.45 s and moves on once the heading has swung back past straight, "proof it actually cancelled the drift, not just a timer"; it then

settles onto the lane centre from the heading and both side TF-Lunas (both walls 8 to 110 cm away, heading within 10°, gain 0.5, at most 18 of steer, 3 confirmations).

- During a pillar pass, a side wall closer than 18 cm (`PILLAR_WALL_WARNING_CM`) adds a correction away from it that grows to 35 at 6 cm; at 10 cm a fixed critical steer takes over, and at 6 cm an emergency steer at speed 65.
- After a corner a side-distance correction recentres the car (`ROUND2_RECENTER_*`: gain 0.45, at most 14, heading within 8°).
- `CORNER_CLEARANCE_RECOVERY` reverses (speed 150, at most 0.4 s) until the front has 30 cm of clearance and re-checks the corner while stopped; it ends in `FAULT` if both side walls are too close for a corner reverse, or if the ESP32 does not report the reverse as applied. `TURN_SIDE_CLEARANCE_CM` is 20 (501 in v0.4.0, which carried 20 as a comment); the v0.4.0 side nudge is gone.

Corners. The corner approach is entered after three fresh front readings of 105 cm or less ("Enter the real corner approach after three fresh geometry samples", `CORNER_ZONE_LOCK_*`); after a corner the recognisers stay disarmed until the centre sensor sees a clearly open straight (130 cm, 3 samples), while the camera keeps detecting pillars. The black-wall check of §9 is a second corner cue. New states: `PRE_TURN_BACKUP` (0.28 s back at speed 88 before a turn) and `POST_TURN_SCAN_HOLD`, a stop after the post-turn reverse "so several stationary frames can prove a nearby pillar". The post-turn reverse is 0.35 s at speed 150 with steering 70 (2.8 s in v0.4.0).

Pillars. Two per straight with the slot-2 timeout, the side masks and the parking-lot rule (§9). Faster than v0.4.0: approach 120, pass 120 (100), release 100 (50). The red target column is at 38 % of the width (25 %) and the near band starts at 3,200 px×px (4,000).

Parking. After the twelfth corner the state `PARKING_PARALLEL_HANDOFF` calls the parking module the switches select (§8). Leaving the parking area at the start adds a 4.5 s `PARKING_EXIT_REVERSE_VIEW` phase and a forward heading alignment (`PARKING_EXIT_FORWARD_ALIGN_*`).

Run log. `RunCsvLogger` can write every run to `run_logs/round2_run_<date_time>.csv`; it is off as committed (`ENABLE_RUN_CSV_LOGGING = False`, §7). `DEBUG = True`.

Values in §1-§6 that no longer hold:

§	v0.4.0	Current
1	drive speed 210	215
1, 5, 6	COMPLETE after the twelfth corner; parking at the end off	hand-over to a parking module after the twelfth corner
3, 5, 6	last-resort front trigger 10 cm (<code>PYTHON_EMERGENCY_RELEASE_CM</code>)	5 cm, deeper inside the TF-Luna's specified 0.2 m minimum
4	post-turn reverse 2.8 s	0.35 s at speed 150, after a 0.28 s reverse before the turn
5	same pillar ignored for 0.6 s after a pass (<code>PILLAR_REACQUIRE_COOLDOWN_SECONDS</code>)	removed; two pillars per straight instead
3	corner reverse at speed 70 for at most 0.8 s; deep recovery at speed 55	200 for at most 0.5 s; 75
2	obstacle corner confirmed on 5 samples in 0.30 s within 8°	3 samples in 0.25 s within 12°

11. Open Challenge firmware

[open-challenge/open-challenge.ino](#) drives the Open Challenge on the ESP32 alone, without the Raspberry Pi or the camera. It uses the same pins, I2C multiplexer channels and sensors as the Obstacle firmware; the servo centre is `SERVO_CENTER = 106` here and 85 in the Obstacle firmware. Constants are in the file.

1. **Start and direction.** After the start button (GPIO32), the car drives at speed 190 (`DETECTION_SPEED`) until one side reading opens beyond 100 cm on 3 samples (`DIRECTION_DETECTION_DISTANCE_CM`, `DIRECTION_CONFIRMATION_SAMPLES`); that side sets clockwise or anticlockwise, keeping the original priority for the right side.

2. **Wall following.** At speed 190 (`FOLLOW_SPEED`) the car holds its heading (gains 0.55 and 0.08, `HEADING_KP`, `HEADING_KD`) and keeps 30 cm from the followed wall (`TARGET_LEFT_DISTANCE_CM`, `TARGET_RIGHT_DISTANCE_CM`; gain 0.50, at most 12), with at most 25 of steering in total.
3. **Corners.** When the followed side opens beyond 75 cm on the left or 50 cm on the right on 3 samples, the car turns 90° on the BNO055 heading at speed 160 (gains 0.75 and 0.06, at most 35 of steering) until the error is under 4°. It then drives at speed 210 for a minimum time (500 ms clockwise, none anticlockwise) and reacquires the wall below 50 cm on 3 samples.
4. **Finish.** After twelve turns (`TOTAL_TURNS`) it holds the heading of the last turn and brakes when the centre TF-Luna reads 150 cm or less (`FINAL_CENTER_DISTANCE_CM`); the electrical brake then stays latched.

The control loop runs every 10 ms (`CONTROL_PERIOD_US`), and servo pulses come from a dedicated task on the other core (`servoPulseTask`, 20 ms frames).

05 Engineering decisions

Each change as a decision: what it replaced, why, the evidence, and the cost (criterion 4).

7. Why each change was made

Each change as a decision: what it replaced, the reason the team gives, the evidence in this repository, and what it costs. The largest test-driven change in this vehicle's line is in the Nationals record: the wheel-shed failure of 2026-08-06 led to the space-frame redesign that every later frame descends from ([1 - Mobility §3, D8](#)).

Change	Instead of	Reason (team)	Evidence here	Trade-off
New 84-step frame, 332 g bare	the Nationals 69-step space-frame	a sturdier structure	build file; the car weighs 862 g against 785 g at Nationals (measured, all changes together)	77 g more to accelerate and stop; stiffness not measured
Steering column turning the whole front axle, 60° each way	single-servo Ackermann	a smaller turning circle	turning circle not measured	the front wheels no longer follow Ackermann geometry, so they scrub in tight turns
Wheels: front 45 mm, rear 55 mm tyres	front 41 mm, rear 55.6 mm	rear tyres for grip	not measured	none recorded
Camera on a 24.8 cm tower, 12° down	a front arm at about 10 cm	a better view of the track	measured mount, derived geometry: pillar bases at 0.5 m and 1.0 m are 12.5° apart against 5.6° before, a 2.2 × stronger range signal (§4)	a 19 cm blind zone ahead of the wheels and a view over the walls (§4); a taller top, its centre-of-gravity height not measured
Six WS2812 LEDs on the front frame	ambient light	fewer colour mistakes	0.11 A at the committed brightness of 80 by the rule of thumb (0.07 A at 50, the bench setting); Buck-2 idle 0.13 A measured with them (§5); misread counts not measured	data at 3.3 V may flicker (§6)
Side rollers fitted, then removed	none	fitted to keep the car moving on wall contact; removed once the rewritten wall-avoidance code made them unnecessary, and because they blocked the Raspberry Pi's ports (reported 2026-09-22)	the team ran the car in both configurations without trouble (team; the number of runs is not recorded); the current <code>main.py</code> adds lane recentering after every pillar pass and after corners, a wall guard during pillar passes and a corner-clearance recovery (software §10)	without rollers a wall contact stops the car; only the Pi's collision escape (§6) remains
Firmware front stop removed (2026-09-10)	a stop under 18 cm on the front TF-Luna	stopping the drive motor ended a competition run	one run ended (team)	collisions are handled by the Pi's distance logic alone (§6)
Parallel parking at the end (<code>parallel_parking.py</code>)	direct entry (<code>partial_parking.py</code>)	15 points for a full parallel park against 7 (2026 rules, scoring element 1.8.2)	rules; no parking attempt logged yet (software §8)	a longer manoeuvre beside the limitations; touching one ends the round (rule 9.24.7)

The 2026-09-10 front-stop decision. The firmware used to stop the drive motor when the front TF-Luna read under 18 cm (released at 24 cm) or when its reading went stale, and that stop ended a run. On 2026-09-10 `forwardHardStopIsActive()` was changed to return `false`, with the comment "Disabled: stopping the drive motor here ends a competition run". Collisions are now handled on the Pi: back away at speed 80 for 0.30 s, require at least 3 cm of progress, at most 2 attempts, release at 30 cm. The team kept the earlier versions as backups, which are not

committed: `obstacle-challenge-final-code_withLED.ino.before-remove-front-brake-20260910.bak` (MD5 `a0f20dd94d2dce1a27c2fc72e3700922`) and `obstacle-challenge-final-code_withLED.ino.before-parking-override-20260910.bak` (MD5 `c93dd67690d2153980442369aa3df41d`). The same day added a `PARKING_OVERRIDE_ON` serial command to the firmware; `main.py` never sends it, so it has no effect in a run.

06 Reproducibility

How to rebuild and run the software exactly as committed (criterion 5).

The code the vehicle runs at WRO Future Engineers APAC 2026: the Obstacle Challenge stack and the Open Challenge firmware. The vehicle is described in [docs/apac_2026_vehicle.md](#), and how `main.py` drives the run, with its state diagrams, in [docs/apac_2026_software.md](#). The two parking modules are described in its [section 8](#).

File	Runs on	What it does
obstacle-challenge/main.py	Raspberry Pi 5	Camera, pillar and line detection, and the driving decisions; sends drive commands to the ESP32
obstacle-challenge/parallel_parking.py	Raspberry Pi 5	Parallel parking at the end of the run (<code>BUILD_ID = "slot-guided-entry-v29"</code>): finds the two parking-lot limitations with the camera, drives into the gap and straightens up. Imports <code>main.py</code> ; called by <code>main.py</code> after the twelfth corner when <code>PARALLEL_PARKING = True</code> , or run on its own
obstacle-challenge/partial_parking.py	Raspberry Pi 5	Direct entry into the parking space on a locked heading, reusing the parallel module's approach and block search. Called by <code>main.py</code> after the twelfth corner when <code>PARTIAL_PARKING = True</code> (as committed), or run on its own
obstacle-challenge/esp32/obstacle-challenge-final-code/obstacle-challenge-final-code.ino	ESP32	Drive motor, steering servo, LEDs, the three TF-Luna sensors and the BNO055 through the PCA9548A; sends telemetry to the Pi
open-challenge/open-challenge.ino	ESP32	The Open Challenge on the ESP32 alone: direction detection, wall following with heading hold, twelve 90° turns on the BNO055 heading, and a final stop at 150 cm from the wall ahead (software §11)
obstacle-challenge/parking_camera_calibration.json	Raspberry Pi 5	The team's image-to-floor calibration for the parking modules (1280 × 720 image, 100 × 150 cm rectangle)
tools/lab-calibration.py	Pi or laptop	Interactive Lab colour-threshold picker (trackbars), with the same image preprocessing as <code>main.py</code> ; how it is used at a venue is in vehicle §4
tools/camera-settings.py	Pi or laptop	Interactive brightness, contrast and gamma preview

Serial link

115200 baud. `main.py` opens `/dev/ttyUSB0` on the Pi (`COM3` is its Windows bench setting).

- Pi to ESP32: `speed,direction,steer` and a newline. `speed` is the 8-bit PWM duty (0 to 255); `steer` is a correction of -60 to +60 around the servo centre of 85.
- ESP32 to Pi: telemetry lines `T,seq,ms,heading,left,center,right,validMask,hardStop,appliedSpeed`.

Settings as committed

Setting	Value	File
Drive speed	<code>SPEED = 215</code> (84 % duty; 210 in v0.4.0)	<code>main.py</code>
Turning, parking, collision escape	<code>100, 70, 80</code>	<code>main.py</code>
Leave the parking area at the start	<code>ENABLE_PARKING_EXIT = True</code>	<code>main.py</code>
Park at the end	<code>ENABLE_PARKING_IN = True</code> : after the twelfth corner <code>main.py</code> hands over to a parking module, chosen by <code>PARTIAL_PARKING = True</code> and <code>PARALLEL_PARKING = False</code> as committed (exactly one must be on; software §8)	<code>main.py</code>

Setting	Value	File
Run log	<code>ENABLE_RUN_CSV_LOGGING = False</code> ; set to <code>True</code> , every run writes <code>run_logs/round2_run_<date_time>.csv</code> beside <code>main.py</code> , one row per telemetry sample and one per event	<code>main.py</code>
Parking modules	run on their own, e.g. <code>python parallel_parking.py --direction clockwise --last-pillar auto</code>	<code>parallel_parking.py</code> , <code>partial_parking.py</code>
Parking run log (CSV and video per run)	<code>ENABLE_RUN_LOGGING = False</code>	<code>parallel_parking.py</code>
Servo centre	<code>SERVO_CENTER = 85</code> in the Obstacle firmware; 106 in the Open Challenge firmware	firmware
Mux channels	TF-Luna left 0, centre 3, right 2; BNO055 4	firmware
Front hard stop	disabled since 2026-09-10 (<code>forwardHardStopIsActive()</code> returns <code>false</code>); see vehicle §7	firmware

Both parking modules read `parking_camera_calibration.json` beside them. It depends on how the camera is mounted: the team's file, made on the practice mat with the September camera mount, is committed; `python parallel_parking.py --calibrate` makes a new one if the mount changes.

On the Pi, `main.py` sets GPIO3 high once it has initialised and is waiting for the start signal, and low when it exits.

Provenance and checksums

Copied on 2026-09-21 from the team coach's working repository (private; commit `63c2b31`). The two parking modules arrived on 2026-09-23 as a Google Drive download (`drive-download-20260922T184733Z-1-001.zip`, which dates them 2026-09-22 03:49 and 04:26) and are stored unchanged. The ESP32 file was `obstacle-challenge-final-code_withLED.ino`; it is renamed here and placed in a folder of the same name, as the Arduino IDE expects. The files are stored byte-for-byte (see [.gitattributes](#)), so these MD5s can be checked on any checkout:

File	MD5
<code>main.py</code>	<code>77187817b99846d4c7d8766d5e9b8bf7</code>
<code>obstacle-challenge-final-code.ino</code>	<code>a0bf8c1ca5330442bed69d8299892572</code>
<code>lab-calibration.py</code>	<code>aad32eb3ca1954ad38683ee5a004da1a</code>
<code>camera-settings.py</code>	<code>f02e7f9eaae329e8867e965ac4f98511</code>
<code>parallel_parking.py</code>	<code>9d622df00da023df403fc06669c3c014</code>
<code>partial_parking.py</code>	<code>34bb36075c2ab680220bd6557056992a</code>
<code>open-challenge.ino</code>	<code>caf69d495f3d0f259a796dfcb81e5b5c</code>
<code>parking_camera_calibration.json</code>	<code>d2ebad50bf7a573496625ee7d20f26da</code>

`main.py` was replaced on 2026-09-23 by the version dated 2026-09-18, from a second Google Drive download (`drive-download-20260922T190246Z-1-001.zip`) whose `cmd.txt` shows the coach starting `final-code/obstacle-challenge-final-code/main.py`. The version it replaces (MD5 `238dd1af810112d09598e433b483d5f8`), which [software §1-§6](#) describes, stays in the history ([the file before the change](#)). The ESP32 firmware in those downloads was byte-identical to the file then committed (MD5 `28a6cdfb`). Every `main.py` name the parking modules use also exists in the 2026-09-18 file.

Not included, because none of them runs on this vehicle: the build without LEDs, an older firmware for a smaller robot, backup copies (`.bak`), component tests and the rest of the coaching repository. The Open Challenge variant for the smaller robot is left out for the same reason.

On 2026-09-23 the Obstacle Challenge files were updated again from the coach's working copy (the `FutureEngineers` folder: commit `63c2b31` plus uncommitted changes): `main.py` now hands over to a parking module after the twelfth corner ([software §10](#)), the parking modules and `lab-calibration.py` carry new values, and the ESP32 firmware sets the

LEDs to brightness 80 (50 before). Earlier versions stay in the history. Every `main.py` name the parking modules use exists in the committed `main.py`.

Also left out from the September downloads: earlier and experimental versions of `main.py` (`main_16_Sep.py`, `mainversion2.py`, `corner_slot_filler.py`) and standalone tests (`parking_entry_only.py`, `cornering_sequence.py`). From that working copy, also left out: `main_background_notworking.py` (an experiment its name marks as not working), `parking_exit_only.py` and the `basic-functions/` and `invididual-components/` tests.